

Story 1: Maven Project and the Dependency Ban

Acceptance Criteria

- One Maven project rooted in the sprint folder builds with `mvn clean test-compile` on a machine that has never seen your code.
- `jakarta.validation-api` is the only non-test dependency; Spring, a servlet API, a JDBC driver, MyBatis and a connection pool are absent from the dependency tree, including transitively.

Tasks

Set up the project on Java 21 with JUnit 5 and surefire, sources under `src/main/java` and tests under `src/test/java`, below one base package declared in `manifest.env`.

Ban the forbidden dependencies in your own build with the enforcer plugin's `bannedDependencies` rule, so a failing build tells whoever added one why.

Run `mvn install` so that Sprint 6 can resolve this module from your local Maven repository by the coordinates you chose.

Notes

- Bean Validation annotations are declarations rather than behaviour, which is why they are the one framework the architecture allows inside the domain.

Story 2: Domain Modelling (UML Class and Sequence Diagrams)

Acceptance Criteria

- A class diagram in design/ shows every type you wrote, its fields, the operations that carry behaviour, the enumerations, the exception hierarchy and the relationships with their cardinality.
- A sequence diagram shows one order arriving at the domain and the eight rules being evaluated in order, including where each failure leaves the flow.
- Both diagrams match the code and are walked in both directions at the review.

Tasks

Draw the class diagram of your design, not of the model the brief describes.

Draw the refusal paths on the sequence diagram; an order sailing through eight boxes and coming out accepted has not documented the interesting half.

Commit Mermaid in `design/class-diagram.md` and `design/sequence-diagram.md`, or commit an exported image rather than a link to a cloud document.

Update the diagrams when the code moves.

Story 3: Domain Entities

Acceptance Criteria

- Account, Instrument, Order and Position are implemented and match the domain model.
- A debit that would leave the balance negative is refused before anything is subtracted, and money is decimal at two places rather than a double.
- A position never goes negative and an order reaches exactly one terminal state, with a disallowed transition refused by the order itself.

Tasks

Give Account the cash balance, the holder, the trading status and the only operations that move that balance, plus the answer to whether it can afford an amount.

Carry both account identifiers and keep them distinct: the numeric key is what `accountId` means everywhere in the API and in the JWT claim, and the string reference is what a support call quotes.

Have the account report the version it was loaded at; the domain does not take the lock.

Give Instrument the symbol in the Fauxnance scheme, the display name, the asset class, the currency of quotation and the answer to whether it may be traded.

Make delisting a flag rather than a deleted row, because the order history references the symbol and that history is the audit trail.

Implement the asymmetric average cost rule: a buy recalculates the average across the old holding and the new units, a sell reduces the quantity and leaves the average alone.

Notes

- The limit price is what the customer submitted. The executed price is what the Trade Executor achieves against a live quote in Sprint 7 and does not exist until the order is filled.

Story 4: Domain Enumerations

Acceptance Criteria

- AccountStatus, OrderSide and OrderStatus each hold exactly the contractual literals, no more and no fewer.

Tasks

Create AccountStatus with ACTIVE, SUSPENDED and CLOSED.

Create OrderSide with BUY and SELL.

Create OrderStatus with NEW, FILLED, REJECTED and CANCELLED, spelling CANCELLED with two Ls.

Notes

- These appear in contracts/trade-api.yaml, the database stores the same strings, and Sprint 9 generates its types from that file, so a renamed or extra literal breaks three places at once.
- There is no partial-fill literal, which is why the Trade Executor fills in full or rejects.

Story 5: Order Request DTO with Validation

Acceptance Criteria

- The order request DTO carries the six fields and every constraint the contract fixes on them.
- Every constraint is enforced and tested, including the boundary either side of each limit.

Tasks

Model the DTO on the PlaceOrderRequest schema in contracts/trade-api.yaml, which is binding.

Constrain the account key, the symbol, the side, the quantity, the price and the idempotency key as the contract states.

Validate by annotation or by hand; either is allowed, and what is assessed is that the constraints hold.

Unit Test Execution Paths

1. valid request passes validation
2. null required field is rejected
3. non-positive quantity is rejected
4. non-positive price is rejected
5. a value one step either side of each limit behaves as specified

Notes

- These are business constraints rather than transport constraints, which is why the DTO lives here and not in the Sprint 6 service.

Story 6: Exception Hierarchy

Acceptance Criteria

- Six exception types cover the six specified cases, all descending from a single domain base type.
- The base type carries the catalogue code and not an HTTP status, and the message is the catalogue message and nothing else.

Tasks

Create one type each for account not found, account not active, instrument not found, insufficient funds, insufficient holdings and duplicate order.

Put anything an investigation needs on a typed field of the exception, logged on the server, rather than in the message.

Decide what happens when a quantity or a price is out of range, given that the list of six has no member for it, and be ready to say what happens when the caller is the Trade Executor replaying an order and never ran a validator.

Declare your six exception class names in manifest.env.

Notes

- One code can mean two things and one case can carry two codes, and neither is an accident. Sprint 6 maps the code to an HTTP status in one place and Sprint 7 maps it to a rejection reason on a Kafka event.
- Leaking internal detail in an error body is OWASP A05.

Story 7: Business Rules 1 to 8

Acceptance Criteria

- Business rules 1 to 8 are implemented in the domain rather than in a controller, enforced in the stated order, and the first failure wins.
- A request breaking two rules receives the code of the first: a suspended account holding no cash gets ACC-403 rather than ORD-400.

Tasks

Enforce existence, then active status, then a tradable instrument, then quantity, then price, then affordability on a buy, then held quantity on a sell, then the idempotency key.

Check quantity and price in the rules as well as on the DTO, because the domain has to hold for a caller that never ran a validator.

Leave room for the two rules that carry no code this sprint: cash and position move together or neither moves, and every order is recorded, including a rejected one.

Design a seam that makes the idempotency rule expressible and testable here without a database, since in Sprint 6 the authority is the unique constraint you built in Sprint 3.

Unit Test Execution Paths

1. buy succeeds and updates cash and position
2. buy rejected on insufficient funds
3. sell rejected on insufficient holdings
4. order rejected on an account that does not exist
5. order rejected on an inactive account
6. order rejected on an unknown or non-tradable instrument
7. duplicate idempotency key rejected
8. a request breaking two rules returns the code of the first

Notes

- Rules in a controller get reimplemented by the Trade Executor, the copies drift, and the first drift accepts an order the e

Story 8: Test-First JUnit Suite

Acceptance Criteria

- AccountTest, OrderLogicTest and PlaceOrderRequestValidationTest are all green, with at least 24 tests across the three.
- The commit history shows the test for a rule in an earlier commit than the code that satisfies it.

Tasks

Write AccountTest over status, debit, credit, affordability, the refusal to go negative, and money that does not drift over many operations.

Write OrderLogicTest over each of the eight rules firing and not firing, plus the evaluation order itself.

Write PlaceOrderRequestValidationTest over every DTO constraint, including the boundary either side of each limit.

Commit in small pieces so the history can show the discipline.

Notes

- A single unit test is one class, one method, one execution path; cover error paths as isolated tests.
- A history in which a rule and its test first appear together does not meet the criterion, and one commit at the end of the week holding the whole module fails it outright.